

plan

Helix Ecosystem Deep Analysis & Integration Master Plan

Objective

Perform exhaustive analysis of all Helix repositories and CLI agent codebases, create comprehensive integration plans, technical documentation, and testing strategies for complete incorporation of all power features into HelixAgent and HelixCode.

Stage 1: Repository Discovery & Structure Mapping

Skill: deep-research-swarm (Route A - Wide Search + File Analysis)

1.1 Main Project & Submodule Cloning

- Clone HelixCode main repository
- Identify ALL submodules (declared in .gitmodules)
- Recursively initialize ALL submodules via SSH
- Map complete repository tree and dependency graph

1.2 Submodule Integration Audit

- Check which submodules are already integrated
- Identify missing submodules from the required list
- Verify SSH vs HTTPS usage
- Document submodule relationships and interdependencies

1.3 CLI Agent Discovery

- Explore helix_agent/cli_agents directory completely
- Identify ALL CLI agent implementations
- Map features, APIs, optimizations per agent
- Priority: claude-code-source first

Stage 2: Deep Codebase Analysis (Parallel Swarm)

Skill: deep-research-swarm (Route B - Focused Deep Dive)

2.1 Main Projects Analysis (Parallel Agents)

- **Agent A1:** HelixCode - Main orchestrator, architecture, entry points
- **Agent A2:** HelixAgent - Core agent framework, cli_agents directory
- **Agent A3:** HelixLLM - LLM integration layer, providers, APIs
- **Agent A4:** HelixMemory - Memory systems, storage, retrieval

- **Agent A5:** HelixSpecifier - Spec parsing, requirements engineering
- **Agent A6:** helix_qa - Testing framework, test banks, QA sessions

2.2 Supporting Infrastructure Analysis (Parallel Agents)

- **Agent B1:** LLMsVerifier - Verification logic, model evaluation
- **Agent B2:** Challenges - Test challenges, benchmarking suite
- **Agent B3:** containers - Containerization, deployment, runtime

2.3 CLI Agent Deep Dive (Priority Pipeline)

- **Agent C1:** claude-code-source - PRIMARY (architecture, features, APIs, hacks)
- **Agent C2-Cn:** All other CLI agents in cli_agents directory
- Feature extraction: power features, innovations, optimizations, workarounds
- Performance analysis: what each does better than Helix
- Gap analysis: missing features in Helix ecosystem

Stage 3: Cross-Reference & Gap Analysis

Skill: Orchestrator synthesis (no external skill)

3.1 Feature Matrix Construction

- Build comprehensive feature comparison tables
- Map Helix ecosystem capabilities vs CLI agent capabilities
- Identify ALL gaps, missing features, performance differences

3.2 Integration Point Mapping

- Where each CLI agent feature fits in Helix architecture
- Submodule extension requirements
- API compatibility analysis
- Dependency resolution

Stage 4: Integration Planning & Documentation

Skill: report-writing

4.1 Master Integration Plan

- Phased approach with dependencies
- Fine-grained tasks and subtasks
- Risk assessment per phase
- Rollback strategies

4.2 Technical Documentation

- Architecture diagrams
- API schemas and wireframes
- Step-by-step integration guides
- Submodule wiring documentation

4.3 Testing Strategy

- Test coverage requirements (100%)
- Challenges integration
- helix_qa test suite expansion
- QA session protocols

Stage 5: Submodule Integration & Verification

Skill: vibecoding-general-swarm + custom scripts

5.1 SSH Submodule Integration

- Convert/add ALL submodules via SSH
- Recursive initialization
- Verification of complete tree

5.2 Build & Test Verification

- Build system verification
- Test execution across all submodules
- Challenges execution
- helix_qa sessions

Stage 6: Final Documentation Assembly

Skill: report-writing + docx/pdf

6.1 Complete Documentation Package

- Executive summary
- Technical deep-dive per component
- Integration guides
- Testing protocols
- Diagrams and visual materials

6.2 Delivery

- Markdown version
- Word document (.docx)
- PDF for distribution

Agent Deployment Strategy

Phase 0: Discovery (Sequential)

- Clone and map repositories
- Identify all components

Phase 1: Analysis (Maximum Parallelism)

- Deploy 9+ analysis agents simultaneously
- Each agent focuses on 1-2 repositories
- Extract: architecture, features, APIs, gaps

Phase 2: Synthesis (Sequential with parallel reviewers)

- Cross-reference all findings
- Build feature matrices
- Identify integration points

Phase 3: Planning & Documentation (Parallel writers)

- Integration plan document
- Technical documentation
- Testing strategy
- Visual diagrams

Phase 4: Integration & Verification (Sequential)

- Actual submodule integration
- Build verification
- Test execution

Phase 5: Assembly (Sequential)

- Final documentation compilation
- Format conversion
- Delivery

Output Artifacts

1. `repository_map.md` - Complete tree of all repos
2. `feature_matrix.md` - Comparison of all CLI agents
3. `gap_analysis.md` - Missing features and gaps
4. `integration_plan.md` - Phased integration plan
5. `technical_documentation.md` - Deep technical docs
6. `testing_strategy.md` - Complete testing approach
7. `architecture_diagrams/` - Visual materials
8. `submodule_integration_log.md` - Integration steps executed
9. `Helix_Master_Documentation.docx` - Final compiled document
10. `Helix_Master_Documentation.pdf` - PDF version

Critical Constraints

- ALL submodules MUST use SSH (not HTTPS)
- 100% test coverage requirement
- No feature omissions - every power feature must be documented
- `claude-code-source` is PRIMARY but ALL agents must be processed
- No skipping, omitting, or relativizing any component